Vidyavardhaka Sangha®, Mysore
VIDYAVARDHAKA COLLEGE OF ENGINEERING
Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi
(Approved by AICTE, New Delhi & Government of Karnataka)
Accredited by NBA (CV, CS, EE, EC, IS & ME) | NAAC with 'A' Grade
P.B. No. 206, Gokulam III Stage, Mysuru-570 002, Karnataka, India
Phone: +91 821 4276201 /202 /225, Fax: +91 824 2510677
Web: http://www.vvce.ac.in
@vvceofficial
VVCE

FULL STACK DEVELOPMENT-1 (BCSFA515)

Activity Based Assessment

On

"Music Playlist Creator with Audio Upload & Playback"

Submitted By:

Nandi Prasad K M	**4VV23CS138**
Nikhil Gowda	**4VV23CS139**
Prajwal M	**4VV23CS152**

Submitted To:

Dr. k. Paramesha
Department of
Computer Science and Engineering VVCE

CampusConnect is a full-stack web application designed to manage college events, workshops, and student registrations. The backend is built using **Node.js** and **Express.js**, providing a custom REST API that communicates with a **MongoDB** database. The frontend consumes these APIs to display dynamic content and handle user interactions.

Database Architecture

The application uses **MongoDB**, a NoSQL database, modeled using the **Mongoose** ODM (Object Data Modeling) library. The database is structured into four primary collections:

A. Users Collection (users)

Stores authentication credentials and student profile information.

- **Schema Definition:**
    - username: String (Unique, Required) o email: String (Unique, Required) o password: String (Hashed for security) o role: String (Enum: 'user', 'admin', Default: 'user') o profile: Object containing student details:
        - USN: String (University Seat Number)
        - FullName: String
        - Branch: String
        - Year: String
        - Semester: String
        - PhoneNumber: String

B. Events Collection (events)

Stores all event-related data, including who created it and who is attending.

- **Schema Definition:**
    - title, description, location: Strings (Required) o date: Date object
    - branch: String (Enum: 'cse', 'ise', 'aiml', 'general', etc.)
    - creator: ObjectId (Reference to User - Admin who posted it) o attendees: Array of ObjectIds (References to User - Students who registered)
    - feedback: Array of objects containing:
        - user: Reference to User
        - rating: Number (1-5)
        - comment: String

C. Workshops Collection (workshops)

A parallel collection to Events, specifically for technical workshops.

- **Schema Definition:**

    - It mirrors the Event schema structure (title, date, attendees, feedback, etc.) but is stored separately to allow distinct categorization in the UI.

D. Notifications Collection (notifications) Stores system alerts and

broadcast messages for users.

- **Schema Definition:**

    - user: ObjectId (Reference to User - The recipient) o message: String

    (The notification text)

    - link: String (Optional URL to redirect, e.g., "/events") o read: Boolean

    (Default: false)

Application Programming Interface (API)

The backend exposes RESTful endpoints grouped by functionality. All protected routes require a valid **JSON Web Token (JWT)** in the request header (Authorization: Bearer <token>).

API URLs and Working

The API is organized into logical route files. The server.js file directs all requests starting with /api/... to the correct route file.

1. Auth API (routes/auth.routes.js)

    - POST /api/auth/register o Working: This endpoint is called when a new student registers. It receives the username, email, password, usn, branch, year, and semester from the registration form. o It checks if the email or USN already exists. If not, it creates a new user, hashes their password, and saves them to the users collection.

    - POST /api/auth/login o Working: This endpoint is called on the login page. It finds a user by their email, compares their hashed password, and if successful, returns a JSON Web Token (JWT). This token is saved by the frontend to prove the user is logged in for future requests.

2. User API (routes/user.routes.js)

    - GET /api/users/profile o Working: This is a protected route. It finds the user's ID from their valid token, fetches their full profile from the users collection, and returns it to be displayed on the "My Profile" page.

3. Events API (routes/event.routes.js)

    - GET /api/events o Working: Fetches *all* events from the database, populating (joining) the creator and attendee data. This is used by your main "All Events" page.

    - GET /api/events/myevents

        - Working: Fetches *only* the events where the logged-in user's ID is present in the attendees array. This is used by the "My Registered Events" page.

    - POST /api/events o Working: An Admin-only route. It receives the event details from the "Add Event" form and creates a new document in the events collection.

- POST /api/events/:id/register o Working: A User-only route. When a user clicks "Register" on an event, this route checks if the user is an admin (and blocks them if so). It then adds the user's ID to that event's attendees array and saves it.
- POST /api/events/:id/feedback o Working: A User-only route. It checks if the event date has passed and if the user was an attendee. If so, it adds their rating and comment to the event's feedback array.

4. Workshops API (routes/workshop.routes.js)

- Working: This file is an exact parallel of the Events API. It provides the same functionality but for the workshops collection.
    - GET /api/workshops o GET /api/workshops/myworkshops o POST /api/workshops (Admin only) o POST /api/workshops/:id/register (User only) o POST /api/workshops/:id/feedback (User only)

5. Notifications API (routes/notification.routes.js)

- GET /api/notifications o Working: Fetches all notifications for the *currently logged-in user*.
- GET /api/notifications/admin/all o Working: An Admin-only route that fetches *every* notification in the database, populating the username for the admin's "system log" view.
- POST /api/notifications/broadcast
    - Working: An Admin-only route. It receives a message from the admin, finds all users in the users collection with the role user, and creates a new notification document for each of them.
- DELETE /api/notifications/:id o Working: An Admin-only route that finds a notification by its ID and deletes it from the database.